AAE 590LGM: Lie Group Methods

Problem Set 3

Travis Hastreiter

2 March, 2026

Contents

P1) Normal Subgroups and Quotient Groups **3**
1.b) **Bank-to-Turn, Trajectory, and Simulation:** 3

P2) SE(2) and the Semi-Direct Product **8**
2.c) **Autonomous Error Dynamics:** 8

P3) SE(2) Lie Algebra, Exp/Log, and Adjoint **9**
3.a) **Group Operations:** 9
3.b) **Lie Algebra $\mathfrak{se}_2(2)$:** 9
3.c) **Exponential and Logarithm Maps:** 9
3.d) **Adjoint Ad_X and Small Adjoint ad_ξ:** 9
Implementation and Unit Tests 9

P4) SE(()2) Kinematics Simulation **15**
4.a) and 4.b) **Dynamics as Mixed Invariant on $\mathbf{SE}_2(2)$ and Adding Wind (World-Frame):** 15
4.c) 2D INS Simulator 16

Code Listing

1 SE(2) Fixed-Wing Velocity Kinematics Code 5
2 Unicycle Group Affine Test 8
3 $SE_2(2)$ and $\mathfrak{se}_2(2)$ Function Implementations 9
4 Unit Tests for $SE_2(2)$ Using Pytest 12
5 $SE_2(2)$ Fixed Wing Aircraft Group Affine Test 15
6 2D INS Simulator Code 16

P1) Normal Subgroups and Quotient Groups

A fixed-wing aircraft in coordinated (no-sideslip) flight at constant altitude can be modeled as a rigid body on SE(2), where the state $X \in \mathrm{SE}(2)$ encodes position (x, y) and heading θ. At this level, velocity and yaw rate are treated as direct control inputs.

1.b) Bank-to-Turn, Trajectory, and Simulation:

Figure 1: Turning radius plot

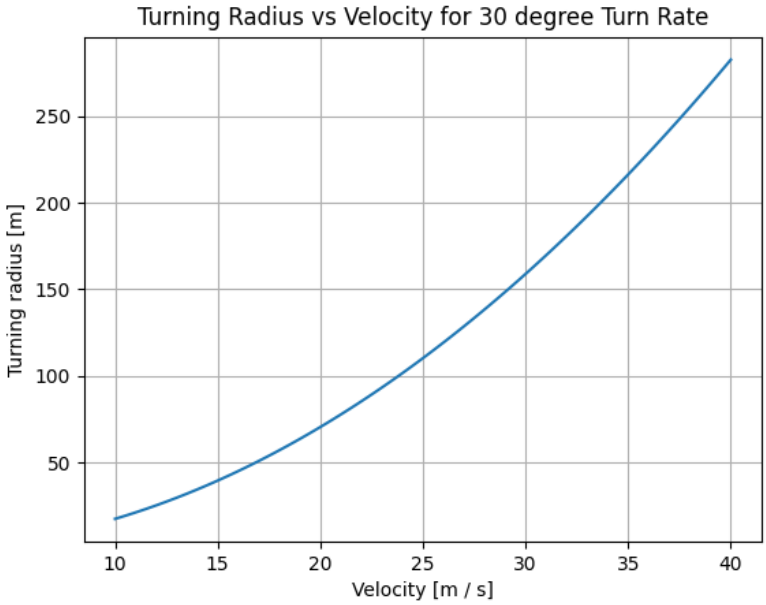


Figure 2: Reference trajectory plot

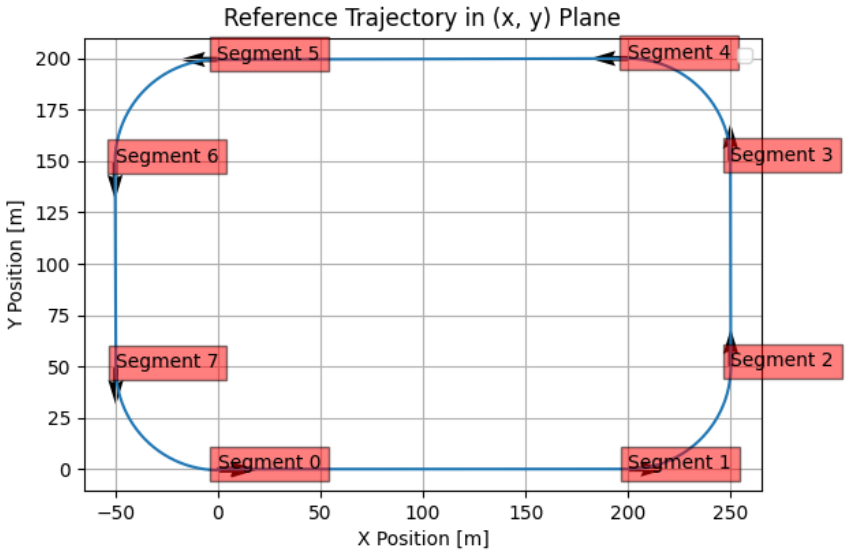


Figure 3: Feasibility check of turn required bank angles

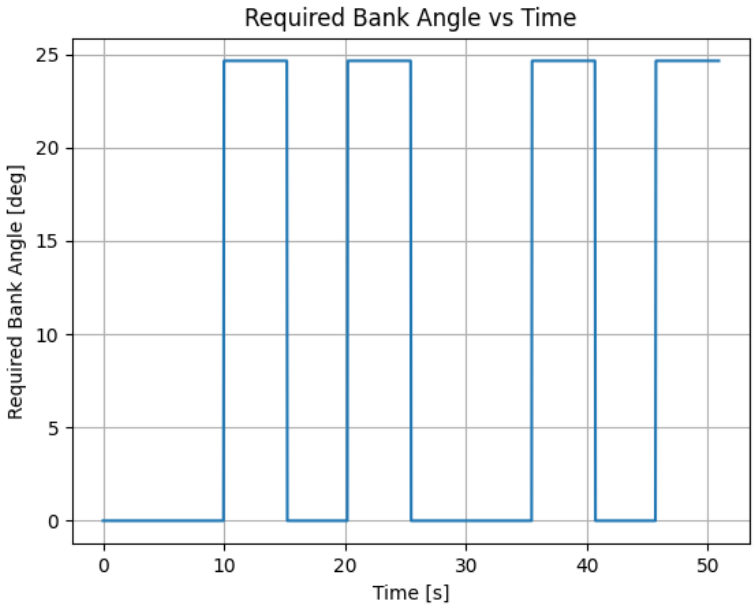
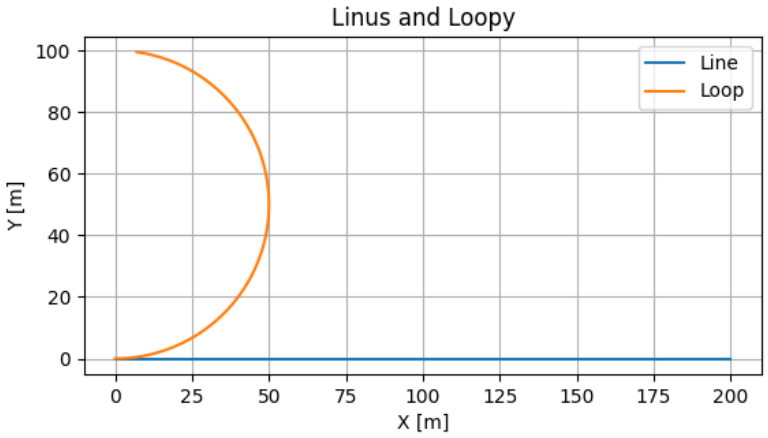


Figure 4: Plots of Linus and Loopy simulation



Code Snippet 1: SE(2) Fixed-Wing Velocity Kinematics Code

```python
import numpy as np
import matplotlib.pyplot as plt
from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator

## 1b.1) Plot turning radius vs velocity
def turning_radius(velocity, desired_turn_rate):
    g = 9.81 # [m / s2]

    turn_radius = velocity ** 2 / (g * np.tan(desired_turn_rate))

    return turn_radius

V = np.linspace(10, 40, 100) # [m / s]
phi_0 = np.deg2rad(30) # Desired turn rate [rad]
turn_radius = turning_radius(V, phi_0)

plt.plot(V, turn_radius)
plt.xlabel("Velocity [m / s]")
plt.ylabel("Turning radius [m]")
plt.title("Turning Radius vs Velocity for 30 degree Turn Rate")
plt.grid()
plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_turn_radius.png")
plt.show()

## 1b.2) Propagate waypoints
X_0 = np.eye(3)

# Segment info
N_seg = 8
v_x = np.array([20, 15, 20, 15, 20, 15, 20, 15]) # [m / s]
v_y = np.zeros_like(v_x)
omega = np.array([0, 0.3, 0, 0.3, 0, 0.3, 0, 0.3]) # [rad / s]
T = np.array([10, 5.24, 5, 5.24, 10, 5.24, 5, 5.24]) # [s]

xi_k = np.array([[v_x], [v_y], [omega]])
```

```
dt = 0.04 # [s]

# Simulation loop
X_k = np.zeros([3, 3, N_seg + 1]) # Waypoint group elements
X_k[:, :, 0] = X_0

T_seg = np.cumsum(T)
time = np.arange(0, T_seg[-1], dt)
k_seg = np.round(T / dt).astype(int)
k_seg_total = sum(k_seg) + 1
X_seg = np.zeros([3, 3, k_seg_total])
X_seg[:, :, 0] = X_0
xi_seg = np.hstack((np.matlib.repmat(xi_k[:, :, 0], 1, k_seg[0]),
                    np.matlib.repmat(xi_k[:, :, 1], 1, k_seg[1]),
                    np.matlib.repmat(xi_k[:, :, 2], 1, k_seg[2]),
                    np.matlib.repmat(xi_k[:, :, 3], 1, k_seg[3]),
                    np.matlib.repmat(xi_k[:, :, 4], 1, k_seg[4]),
                    np.matlib.repmat(xi_k[:, :, 5], 1, k_seg[5]),
                    np.matlib.repmat(xi_k[:, :, 6], 1, k_seg[6]),
                    np.matlib.repmat(xi_k[:, :, 7], 1, k_seg[7])))

for k in range(N_seg):
   X_k[:, :, k + 1] = se2_Lie_group_integrator(X_k[:, :, k], xi_k[:, :, k], T[k])

for s in range(k_seg_total - 1):
    X_seg[:, :, s + 1] = se2_Lie_group_integrator(X_seg[:, :, s], xi_seg[:, s], dt)

# Plot
# Trajectories
plt.plot(X_seg[0, 2, :], X_seg[1, 2, :])
plt.quiver(X_k[0, 2, :], X_k[1, 2, :], X_k[0, 0, :], X_k[1, 0, :])
for k in range(N_seg):
    plt.text(X_k[0, 2, k], X_k[1, 2, k], f"Segment {k}", fontsize=10,
             bbox=dict(facecolor='red', alpha=0.5))
plt.xlabel("X Position [m]")
plt.ylabel("Y Position [m]")
plt.title("Reference Trajectory in (x, y) Plane")
plt.legend()
plt.grid()
plt.gca().set_aspect('equal', adjustable='box')
plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_reftraj.png")
plt.show()

def required_bank_angle(velocity, omega):
    g = 9.81 # [m / s2]

    bank_angle = np.atan(omega * velocity / g)

    return bank_angle

bank_angles = required_bank_angle(xi_seg[0, :], xi_seg[2, :])
plt.plot(time[0:-1], np.rad2deg(bank_angles))
plt.xlabel("Time [s]")
plt.ylabel("Required Bank Angle [deg]")
plt.title("Required Bank Angle vs Time")
plt.grid()
plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_feasck.png")
plt.show()

# Simulate loop and line
dt_loopline = 0.01 # [s]
tf = 10 # [s]
N_t = np.round(tf / dt_loopline).astype(int)
X_k_loop = np.zeros([3, 3, N_t + 1])
X_k_line = np.zeros_like(X_k_loop)
xi_line = np.array([[20], [0], [0]])
xi_loop = np.array([[15], [0], [0.3]])
X_k_loop[:, :, 0] = X_0
```

```
X_k_line[:, :, 0] = X_0

for k in range(N_t):
    X_k_loop[:, :, k + 1] = se2_Lie_group_integrator(X_k_loop[:, :, k], xi_loop, dt_loopline
    )
    a = se2_Lie_group_integrator(X_k_loop[:, :, k], xi_loop, dt)
    X_k_line[:, :, k + 1] = se2_Lie_group_integrator(X_k_line[:, :, k], xi_line, dt_loopline
    )

plt.plot(X_k_line[0, 2, :], X_k_line[1, 2, :], label = "Line")
plt.plot(X_k_loop[0, 2, :], X_k_loop[1, 2, :], label = "Loop")
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.title("Linus and Loopy")
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.grid()
plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_linusloopy.png")
plt.show()
```

P2) SE(2) and the Semi-Direct Product

Group affine systems are a class of nonlinear systems on Lie groups whose error dynamics are **autonomous**—they depend only on the error, not on the absolute state. This property is the theoretical foundation for both geometric control and invariant filtering.

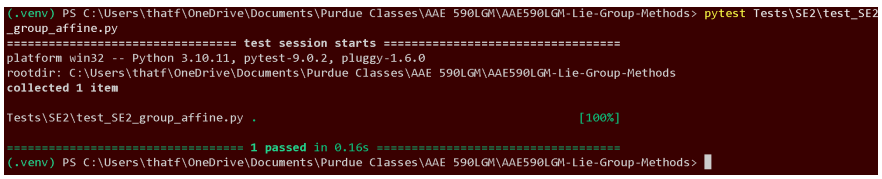
2.c) Autonomous Error Dynamics:

Question:
Verify numerically: Choose random $X, Y \in \mathrm{SE}(2)$ and a twist $\xi = (15, 0, 0.3)^T$. Compute both sides of the group affine equation and confirm agreement to machine precision.
Solution:
Code snippet 1 shows the verification of group affine property of the SE(2) unicycle dynamics

Code Snippet 2: Unicycle Group Affine Test

```python
import pytest as pt
import numpy as np
from Code.SE2.SE2_maps import se2_compose, se2_inverse, se2_exp, se2_log, se2_vee, se2_wedge, se2_Ad, se2_ad

def test_se2_group_affine():
    """
    Test that group affine property holds for random SE2 elements with unicycle dynamics f_u(X) = X xi^
    f_u(XY) = f_u(X) Y + X f_u(Y) - X f_u(I) Y
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi_X_random = np.reshape(rng.uniform(-10, 10, 3 * n_samples), (3, n_samples))
    xi_Y_random = np.reshape(rng.uniform(-10, 10, 3 * n_samples), (3, n_samples))

    xi = np.array([[15], [0], [0.3]])

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X_random = se2_exp(xi_X_random[:, i])
        Y_random = se2_exp(xi_Y_random[:, i])

        lhs = X_random @ Y_random @ se2_wedge(xi)  # f_u(XY)
        rhs = X_random @ se2_wedge(xi) @ Y_random + X_random @ Y_random @ se2_wedge(xi) - X_random @ se2_wedge(xi) @ Y_random # f_u(X) Y + X f_u(Y) - X f_u(I) Y

        checks[i] = np.all(lhs == pt.approx(rhs))

    assert np.all(checks)
```

Figure 5: Group Affine Unicycle Test Passing



P3) SE(2) Lie Algebra, Exp/Log, and Adjoint

In lecture, $\mathrm{SE}_2(3) = \mathrm{SO}(3) \ltimes (\mathbb{R}^3 \times \mathbb{R}^3)$ was introduced for 3D inertial navigation, combining orientation, velocity, and position into a single Lie group. You will now derive the **2D analog**, $\mathrm{SE}_2(2) = \mathrm{SO}(2) \ltimes (\mathbb{R}^2 \times \mathbb{R}^2)$, which adds velocity to the SE(2) pose.

3.a) Group Operations:

Question:
Implement $\mathrm{SE}_2(2)$ compose and inverse
Solution:
Code snippet 3 shows the implementation of `se22_compose(X1, X2)` and `se22_inverse(X)` Code snippet 4 shows its unit test and Fig 6 shows its passing.

3.b) Lie Algebra $\mathfrak{se}_2(2)$:

Question:
Implement `se22_wedge(xi)` and `se22_vee(Xi)`
Solution:
Code snippet 3 shows the implementation of `se22_wedge(xi)` and `se22_vee(Xi)` Code snippet 4 shows its unit test and Fig 6 shows its passing.

3.c) Exponential and Logarithm Maps:

Question:
Implement `se22_exp(xi)` and `se22_log(X)`. Verify round-trip: $\mathrm{Exp}()\big((\mathrm{Log}(()X))^\wedge\big) = X$ for at least 3 random $X \in \mathrm{SE}_2(()2)$ (i.e., `se22_exp(se22_log(X))` recovers X to machine precision).
Solution:
Code snippet 3 shows the implementation of `se22_exp(xi)` and `se22_log(X)`.
Code snippet 4 shows its unit test and Fig 6 shows its passing.

3.d) Adjoint Ad_X and Small Adjoint ad_ξ:

Question:
Implement `se22_Ad(X)` and `se22_ad(xi)`. Verify numerically (for at least 3 random inputs each):

- $\mathrm{Ad}_{X_1 X_2} = \mathrm{Ad}_{X_1} \mathrm{Ad}_{X_2}$
- $[\xi_1, \xi_2] = \mathrm{ad}_{\xi_1} \xi_2$

Solution:
Code snippet 3 shows the implementation of the functions `se22_Ad(X)` and `se22_ad(xi)`. Code snippet 4 shows the implementation of the unit test verifying $\mathrm{Ad}_{X_1 X_2} = \mathrm{Ad}_{X_1} \mathrm{Ad}_{X_2}$ for random $X_1, X_2 \in \mathrm{SE}_2(2)$ for random $X \in \mathrm{SE}_2(2)$ named `test_se22_Ad_composition` and Fig 6 shows the test passing. Code snippet 4 shows the implementation of the unit test verifying $[\xi_1, \xi_2] = \mathrm{ad}_{\xi_1} \xi_2$ for random $\xi_1, \xi_2 \in \mathfrak{se}_2(2)$ named `test_se22_ad` and Fig 6 shows the test passing.

Implementation and Unit Tests

Code Snippet 3: $\mathrm{SE}_2(2)$ and $\mathfrak{se}_2(2)$ Function Implementations

```
import numpy as np
from Code.SO2.SO2_maps import so2_wedge, so2_vee, so2_exp, so2_log

def se22_compose(X1, X2):
    """
    Composition of elements of SE2(2)
```

```
7
8       :param X1: element of SE2(2)
9       :param X2: element of SE2(2)
10      """
11      X1_t = X1[0:2, 3].reshape((2, 1))
12      X1_v = X1[0:2, 2].reshape((2, 1))
13      X1_R = X1[0:2, 0:2]
14
15      X2_t = X2[0:2, 3].reshape((2, 1))
16      X2_v = X2[0:2, 2].reshape((2, 1))
17      X2_R = X2[0:2, 0:2]
18
19      X12_R = X1_R @ X2_R
20      X12_v = X1_R @ X2_v + X1_v
21      X12_t = X1_R @ X2_t + X1_t
22
23      X12 = np.block([[X12_R, X12_v, X12_t],
24                      [np.zeros((2, 2)), np.eye(2)]])
25
26      return X12
27
28  def se22_inverse(X):
29      """
30      Inverse of SE2(2) element
31
32      :param X: element of SE2(2)
33      """
34      X_t = X[0:2, 3].reshape((2, 1))
35      X_v = X[0:2, 2].reshape((2, 1))
36      X_R = X[0:2, 0:2]
37
38      X_inv_t = -X_R.T @ X_t
39      X_inv_v = -X_R.T @ X_v
40      X_inv_R = X_R.T
41
42      X_inv = np.block([[X_inv_R, X_inv_v, X_inv_t],
43                        [np.zeros((2, 2)), np.eye(2)]])
44
45      return X_inv
46
47  def se22_wedge(xi):
48      """
49      Maps from se22 real number parametrization to se22 Lie algebra
50
51      :param xi: twist vector (a_x, a_y, b_x, b_y, omega) in R3
52      """
53      xiwedge = np.block([[so2_wedge(xi[4]), xi[0:2].reshape((2, 1)), xi[2:4].reshape((2, 1))
        ],
54                          [np.zeros((2, 2)), np.zeros((2, 2))]])
55
56      return xiwedge
57
58  def se22_vee(xiwedge):
59      """
60      Maps from se22 Lie algebra to its real number parametrization
61
62      :param xiwedge: 3x3 se22 Lie algebra element
63      """
64
65      xi = np.block([xiwedge[[0, 1], [2, 2]], xiwedge[[0, 1], [3, 3]], so2_vee(xiwedge[0:2,
        0:2])]);
66
67      return xi
68
69  def se22_exp(xi):
70      """
71      Maps from parametrization of se22 Lie algebra to SE22 Lie group
72
```

```
    :param xi: twist vector (a_x, a_y, b_x, b_y, omega) in R3
    """
    # Extract elements
    a = xi[0:2]
    b = xi[2:4]
    omega = xi[4]

    # SO2 exponential map
    R = so2_exp(omega)

    # Translation-orientation coupling
    if abs(omega) > 1e-8:
        V = np.sin(omega) / omega * np.eye(2) + (1 - np.cos(omega)) / omega * np.array([[0, -1], [1, 0]])
    else:
        V = ((1 - omega ** 2 / 6 + omega ** 4 / 120) * np.eye(2)
           + (omega / 2 - omega ** 3 / 24) * np.array([[0, -1], [1, 0]]))

    X = np.block([[R, V @ a.reshape((2, 1)), V @ b.reshape((2, 1))],
                  [np.zeros((2, 2)),     np.eye(2)]])

    return X

def se22_log(xiwedge):
   """
   Maps from SE22 Lie group to the parametrization of its Lie algebra se22

   :param xiwedge: 3x3 se22 Lie algebra element
   """
   omega = so2_log(xiwedge[0:2, 0:2])

   if abs(omega) > 1e-8:
       V_inv = omega / 2 * (np.cos(omega / 2) / np.sin(omega / 2) * np.eye(2) + np.array([[0, 1], [-1, 0]]))
   else:
       V_inv = 1 / 2 * ((2 - omega ** 2 / 6 - omega ** 4 / 360 - 2 * omega ** 6 / 30240 + omega ** 8 / 604800) * np.eye(2) + omega * np.array([[0, -1], [1, 0]]))

   a = V_inv @ xiwedge[0:2, 2]
   b = V_inv @ xiwedge[0:2, 3]

   xi = np.vstack((a.reshape((2, 1)), b.reshape((2, 1)), np.reshape(omega, (1,1))))

   return xi

def se22_Ad(X):
    """
    Maps from se22 to se22 defined by (Ad_X xi)**wedge = X xi**wedge X**-1
    Shifts tangent spaces

    :param X: SE22 Lie group element
    """
    X_t = X[0:2, 3].reshape((2, 1))
    X_v = X[0:2, 2].reshape((2, 1))
    X_R = X[0:2, 0:2]

    J = np.array([[0, -1], [1, 0]])
    v_circ = -J @ X_v
    t_circ = -J @ X_t

    Ad = np.block([[X_R, np.zeros((2, 2)), v_circ],
                   [np.zeros((2, 2)), X_R, t_circ],
                   [np.zeros((1, 4)),           1]])

    return Ad

def se22_ad(xi):
    """
```

```
    Derivative of Adjoint at identity

    :param xi: se2(2) Lie algebra element parametrization
    """
    a = xi[0:2].reshape((2, 1))
    b = xi[2:4].reshape((2, 1))
    omega = xi[4]

    J = np.array([[0, -1], [1, 0]])
    a_circ = -J @ a
    b_circ = -J @ b

    omega_wedge = so2_wedge(omega)

    ad = np.block([[omega_wedge, np.zeros((2, 2)), a_circ],
                   [np.zeros((2, 2)), omega_wedge, b_circ],
                   [np.zeros((1, 4)),                   0]])

    return ad
```

Code Snippet 4: Unit Tests for $SE_2(2)$ Using Pytest

```
import pytest as pt
import numpy as np
from Code.SE22.SE22_maps import se22_compose, se22_inverse, se22_exp, se22_log, se22_vee,
    se22_wedge, se22_Ad, se22_ad

def test_se22_exp_log():
    """
    Test that exp(log(X)) = X for random SE22 elements
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X_random = se22_exp(xi_random[:, i])

        X_log = se22_log(X_random)
        X_tilde = se22_exp(X_log)

        checks[i] = np.all(X_tilde == pt.approx(X_random))

    assert np.all(checks)

def test_se22_Ad():
    """
    Test that Ad_X(xi) = (X xi^wedge X^-1)^vee for random SE22, se22 elements
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
    xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X_random = se22_exp(xi_X_random[:, i])

        xi_prime = se22_vee(X_random @ se22_wedge(xi_random[:, i]) @ se22_inverse(X_random))
        xi_prime_ad = se22_Ad(X_random) @ xi_random[:, i]

        checks[i] = np.all(xi_prime_ad == pt.approx(xi_prime))

    assert np.all(checks)

```

```python
def test_se22_Ad_composition():
    """
    Test that Ad_{X_1 X_2} = Ad_{X_1}Ad_{X_2} for random SE22 elements
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi1_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
    xi2_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X1_random = se22_exp(xi1_random[:, i])
        X2_random = se22_exp(xi2_random[:, i])

        Ad_X1X2 = se22_Ad(se22_compose(X1_random, X2_random))
        Ad_X1_Ad_X2 = se22_Ad(X1_random) @ se22_Ad(X2_random)

        checks[i] = np.all(Ad_X1X2 == pt.approx(Ad_X1_Ad_X2))

    assert np.all(checks)

def test_se22_Ad_inv():
    """
    Test that (Ad_X)^-1 = Ad_{X^-1} for random SE22 elements
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X_random = se22_exp(xi_random[:, i])

        Ad_X_inv = np.linalg.inv(se22_Ad(X_random))
        Ad_Xinv = se22_Ad(se22_inverse(X_random))

        a = se22_Ad(X_random) @ Ad_X_inv
        b = se22_Ad(X_random) @ Ad_Xinv

        checks[i] = np.all(Ad_X_inv == pt.approx(Ad_Xinv))

    assert np.all(checks)

def test_se22_ad():
    """
    Test that ad_{xi_1}(xi_2) = [xi_1, xi_2] for random se22 elements
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi1_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
    xi2_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        Lie_bracket = se22_vee(se22_wedge(xi1_random[:, i]) @ se22_wedge(xi2_random[:, i])
                               - se22_wedge(xi2_random[:, i]) @ se22_wedge(xi1_random[:, i]))
        Lie_bracket_ad = se22_ad(xi1_random[:, i]) @ xi2_random[:, i]

        checks[i] = np.all(Lie_bracket_ad == pt.approx(Lie_bracket))

    assert np.all(checks)
```

Figure 6: Pytest unit test output showing tests passing

```
(.venv) PS pytest Tests\SE22\test_SE22.py          Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods>
============================================= test session starts =============================================
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\thatf\OneDrive\Documents\Purdue Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods
collected 5 items

Tests\SE22\test_SE22.py .....                                                                        [100%]

============================================== 5 passed in 0.28s ==============================================
```

P4) SE(()2) Kinematics Simulation

A 2D fixed-wing aircraft carries a "2D IMU" that measures body-frame angular velocity ω and body-frame acceleration $\mathbf{a} = (a_x, a_y)^T$. Your task is to propagate the aircraft's state (orientation, velocity, position) using the $\mathrm{SE}_2(2)$ group structure—a 2D analog of the 3D IMU navigation problem from the $\mathrm{SE}_2(3)$ lecture.

4.a) and 4.b) Dynamics as Mixed Invariant on $\mathrm{SE}_2(2)$ and Adding Wind (World-Frame):

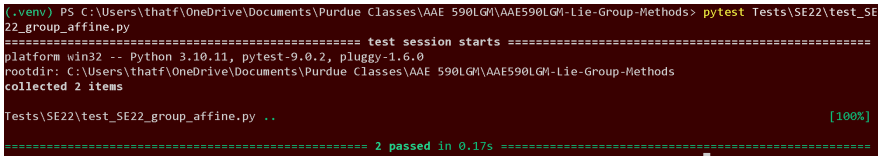
Question:
Test group affine property for fixed wing aircraft dynamics with and without wind
Solution:
Code snippet 5 shows the verification of group affine property of the $\mathrm{SE}_2(2)$ windy fixed wing aircraft dynamics

Code Snippet 5: $\mathrm{SE}_2(2)$ Fixed Wing Aircraft Group Affine Test

```
import pytest as pt
import numpy as np
from Code.SO2.SO2_maps import so2_wedge
from Code.SE22.SE22_maps import se22_compose, se22_inverse, se22_exp, se22_log, se22_vee,
    se22_wedge, se22_Ad, se22_ad

def test_se22_group_affine():
    """
    Test that group affine property holds for random SE2 elements with mixed invariant
    dynamics f_u(X) = xi_R_wedge X + X xi_L_wedge
    f_u(XY) = f_u(X) Y + X f_u(Y) - X f_u(I) Y
    """
    n_samples = 100

    rng = np.random.default_rng()
    xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
    xi_Y_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
    a = np.reshape(rng.uniform(-10, 10, 2 * n_samples), (2, n_samples))
    omega = rng.uniform(-10, 10, n_samples)

    checks = np.zeros([n_samples, 1])
    for i in range(n_samples):
        X_random = se22_exp(xi_X_random[:, i])
        Y_random = se22_exp(xi_Y_random[:, i])

        M = np.zeros((4, 4))
        N = np.block([[so2_wedge(omega[i]), a[:, i].reshape((2, 1)), np.zeros((2, 1))], [np.
    zeros((2, 4))]])
        C = np.block([np.zeros((4, 3)), np.array((0, 0, 1, 0)).reshape((4, 1))])
        xi_R_wedge = M - C
        xi_L_wedge = N + C

        lhs = xi_R_wedge @ X_random @ Y_random + X_random @ Y_random @ xi_L_wedge # f_u(XY)
        rhs = (xi_R_wedge @ X_random + X_random @ xi_L_wedge) @ Y_random + X_random @ (
    xi_R_wedge @ Y_random + Y_random @ xi_L_wedge) - X_random @ (xi_R_wedge + xi_L_wedge) @
    Y_random # f_u(X) Y + X f_u(Y) - X f_u(I) Y

        checks[i] = np.all(lhs == pt.approx(rhs))

    assert np.all(checks)

def test_se22_group_affine_wind():
    """
    Test that group affine property holds for random SE2 elements with mixed invariant
    dynamics with wind f_u(X) = xi_R_wedge X + X xi_L_wedge
    f_u(XY) = f_u(X) Y + X f_u(Y) - X f_u(I) Y
    """
    n_samples = 100
```

```
43
44      rng = np.random.default_rng()
45      xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
46      xi_Y_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
47      a = np.reshape(rng.uniform(-10, 10, 2 * n_samples), (2, n_samples))
48      omega = rng.uniform(-10, 10, n_samples)
49
50      w_a = np.array((3, -2,)).reshape((2, 1)) # [m / s2]
51
52      checks = np.zeros([n_samples, 1])
53      for i in range(n_samples):
54          X_random = se22_exp(xi_X_random[:, i])
55          Y_random = se22_exp(xi_Y_random[:, i])
56
57          M = np.block([np.zeros((4, 2)), np.vstack((w_a, np.zeros((2, 1))))], np.zeros((4, 1))
    ])
58          N = np.block([[so2_wedge(omega[i]), a[:, i].reshape((2, 1)), np.zeros((2, 1))], [np.
    zeros((2, 4))]])
59          C = np.block([np.zeros((4, 3)), np.array((0, 0, 1, 0)).reshape((4, 1))])
60          xi_R_wedge = M - C
61          xi_L_wedge = N + C
62
63          lhs = xi_R_wedge @ X_random @ Y_random + X_random @ Y_random @ xi_L_wedge # f_u(XY)
64          rhs = (xi_R_wedge @ X_random + X_random @ xi_L_wedge) @ Y_random + X_random @ (
    xi_R_wedge @ Y_random + Y_random @ xi_L_wedge) - X_random @ (xi_R_wedge + xi_L_wedge) @
    Y_random # f_u(X) Y + X f_u(Y) - X f_u(I) Y
65
66          checks[i] = np.all(lhs == pt.approx(rhs))
67
68      assert np.all(checks)
```

Figure 7: Windless and Windy Fixed Wing Aircraft Group Affine Test Passing



4.c) 2D INS Simulator

Code Snippet 6: 2D INS Simulator Code

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from Code.SE22.SE22_maps import se22_compose, se22_exp, se22_log, se22_wedge, se22_vee,
       se22_inverse
4  from Code.SE22.SE22_integrators import se22_Lie_group_integrator
5
6  def turning_radius(velocity, desired_turn_rate):
7      g = 9.81 # [m / s2]
8
9      turn_radius = velocity ** 2 / (g * np.tan(desired_turn_rate))
10
11     return turn_radius
12
13 ## 1b.2) Propagate waypoints
14 X_0 = np.eye(4)
15
16 # Segment info
17 dt = 0.01 # [s]
18 N_seg = 4
19 a = np.array([[2, 0, 0,-1],
20               [0, 0, 0, 0]]) # [m / s]
```

```python
omega = np.array([0, 0, 0.3, 0]).reshape([1, 4]) # [rad / s]
T = np.array([5, 5, np.round(np.pi / 0.3 / dt) * dt, 5]) # [s]

xi_k = np.vstack((a, np.zeros_like(a), omega)).reshape([5, 1, -1])

# Simulation loop
X_k = np.zeros([4, 4, N_seg + 1]) # Waypoint group elements
X_k[:, :, 0] = X_0

T_seg = np.cumsum(T)
time = np.arange(0, T_seg[-1], dt)
k_seg = np.round(T / dt).astype(int)
k_seg_total = sum(k_seg) + 1
X_seg = np.zeros([4, 4, k_seg_total])
X_seg[:, :, 0] = X_0
xi_seg = np.hstack((np.matlib.repmat(xi_k[:, :, 0], 1, k_seg[0]),
                    np.matlib.repmat(xi_k[:, :, 1], 1, k_seg[1]),
                    np.matlib.repmat(xi_k[:, :, 2], 1, k_seg[2]),
                    np.matlib.repmat(xi_k[:, :, 3], 1, k_seg[3])))

for k in range(N_seg):
   xi = xi_k[:, :, k]
   if xi[4] != 0: # Add centripital acceleration
       rvec = X_k[0:2, 3, k].reshape([2, 1])
       r = np.linalg.norm(rvec)
       vvec = X_k[0:2, 2, k].reshape([2, 1])
       v = np.linalg.norm(vvec)
       vhat = vvec / v
       omega = xi[4]
       r_turn = turning_radius(v, omega)
       R = X_k[0:2, 0:2, k].T
       a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ R @ vhat
       xi[0:2] = xi[0:2] + a_centrip
   X_k[:, :, k + 1] = se22_Lie_group_integrator(X_k[:, :, k], xi, T[k])
   xi_kc = xi_k[:, :, k]
   X_kc = X_k[:, :, k]
   X_kp1 = X_k[:, :, k + 1]

   hiii = 1

for s in range(k_seg_total - 1):
    xi = xi_seg[:, s].reshape([5, 1])
    if xi[4] != 0: # Add centripital acceleration
       rvec = X_seg[0:2, 3, s].reshape([2, 1])
       r = np.linalg.norm(rvec)
       vvec = X_seg[0:2, 2, s].reshape([2, 1])
       v = np.linalg.norm(vvec)
       vhat = vvec / v
       omega = xi[4]
       r_turn = turning_radius(v, omega)
       R = X_seg[0:2, 0:2, s].T
       a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ R @ vhat
       xi[0:2] = xi[0:2] + a_centrip
    X_seg[:, :, s + 1] = se22_Lie_group_integrator(X_seg[:, :, s], xi, dt)

# Plot
# Trajectories
plt.plot(X_seg[0, 3, :], X_seg[1, 3, :])
# plt.quiver(X_k[0, 3, :], X_k[1, 3, :], X_k[0, 2, :], X_k[1, 2, :])
# for k in range(N_seg):
#     plt.text(X_k[0, 3, k], X_k[1, 3, k], f"Segment {k}", fontsize=10,
#              bbox=dict(facecolor='red', alpha=0.5))
plt.xlabel("X Position [m]")
plt.ylabel("Y Position [m]")
plt.title("Flight Profile in (x, y) Plane")
plt.legend()
plt.grid()
plt.gca().set_aspect('equal', adjustable='box')
```

```
plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_traj.png")
plt.show()

vel_mag = np.linalg.vector_norm(X_seg[0:2, 2, :], axis = 0)
plt.plot(time, vel_mag[0:-1])
plt.grid()
plt.title("Velocity vs Time")
plt.xlabel("Time [s]")
plt.ylabel("Velocity [m / s]")
plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_vel.png")
plt.show()

heading = np.zeros([k_seg_total - 1, 1])
for k in range(k_seg_total - 1):
    xi_recover = se22_log(X_seg[:, :, k])
    heading[k] = xi_recover[4]
plt.plot(time, np.rad2deg(heading))
plt.grid()
plt.title("Heading vs Time")
plt.xlabel("Time [s]")
plt.ylabel("Heading [deg]")
plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_head.png")
plt.show()

plt.plot(time, X_seg[0, 3, 0:-1])
plt.plot(time, X_seg[1, 3, 0:-1])
plt.grid()
plt.title("Position vs Time")
plt.xlabel("Time")
plt.xlabel("Position")
plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_pos.png")
plt.show()

# Simulate spiral
dt_spiral = 0.01 # [s]
tf = 10 # [s]
N_t = np.round(tf / dt_spiral).astype(int)
X_k_spiral = np.zeros([4, 4, N_t + 1])
omega_spiral = 0.5 # [rad / s]
a_spiral = np.array([[1], [0]])
xi_spiral = np.block([[a_spiral], [np.zeros((2, 1))], [omega_spiral]]).reshape([5, 1])
X_k_spiral[:, :, 0] = X_0

for k in range(N_t):
    xi = xi_spiral
    if xi[4] != 0: # Add centripital acceleration
       rvec = X_k_spiral[0:2, 3, k].reshape([2, 1])
       r = np.linalg.norm(rvec)
       vvec = X_k_spiral[0:2, 2, k].reshape([2, 1])
       v = np.linalg.norm(vvec)
       if v < 1e-5:
           vhat = np.zeros([2, 1])
           r_turn = 1
       else:
           vhat = vvec / v
           omega = xi[4]
           r_turn = turning_radius(v, omega)
       R = X_k_spiral[0:2, 0:2, k].T
       a_centrip = np.sign(xi[4]) * v ** 2 / r_turn * np.array([[0, -1], [1, 0]]) @ R @ vhat
       xi[0:2] = xi[0:2] + a_centrip
    X_k_spiral[:, :, k + 1] = se22_Lie_group_integrator(X_k_spiral[:, :, k], xi, dt_spiral)

plt.plot(X_k_spiral[0, 3, :], X_k_spiral[1, 3, :], label = "Line")
v_ind = np.round(np.linspace(0, N_t, 10)).astype(int)
plt.quiver(X_k_spiral[0, 3, v_ind], X_k_spiral[1, 3, v_ind], X_k_spiral[0, 2, v_ind],
    X_k_spiral[1, 2, v_ind])
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
```

```
156 plt.title("Spiral")
157 plt.legend()
158 plt.gca().set_aspect('equal', adjustable='box')
159 plt.grid()
160 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_spiral.png")
161 plt.show()
162
163 vel_mag = np.linalg.vector_norm(X_k_spiral[0:2, 2, :], axis = 0)
164 plt.plot(np.linspace(0, tf, N_t + 1), vel_mag)
165 plt.grid()
166 plt.title("Velocity vs Time for Spiral")
167 plt.xlabel("Time [s]")
168 plt.xlabel("Velocity [m / s]")
169 plt.savefig("./HW3/Parts/Q4/AAE590LGM_HW3_Q4_spiralvel.png")
170 plt.show()
171
172 hi = 1
```

Figure 8: Plots of trajectory

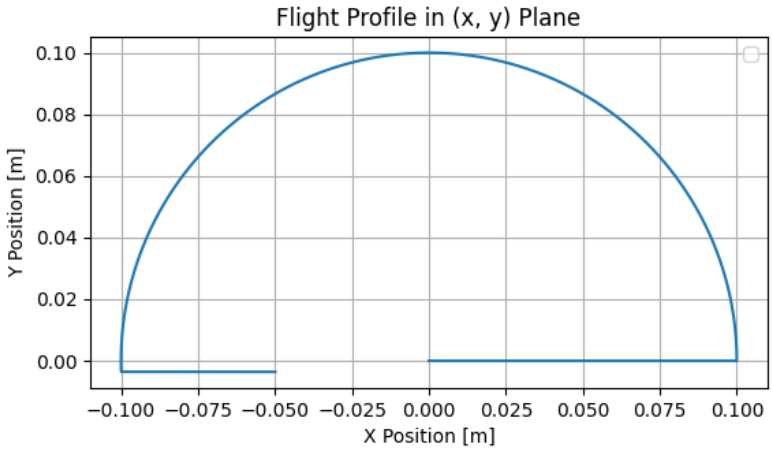


Figure 9: Plots of velocity

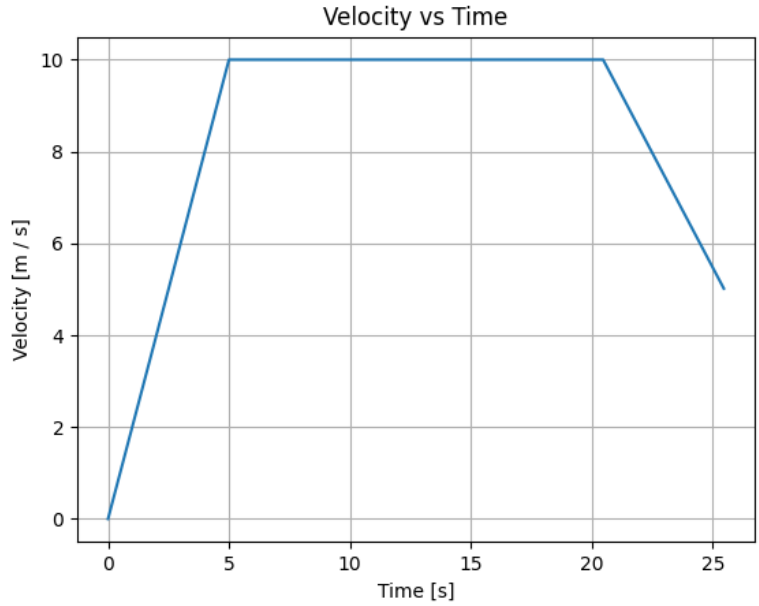
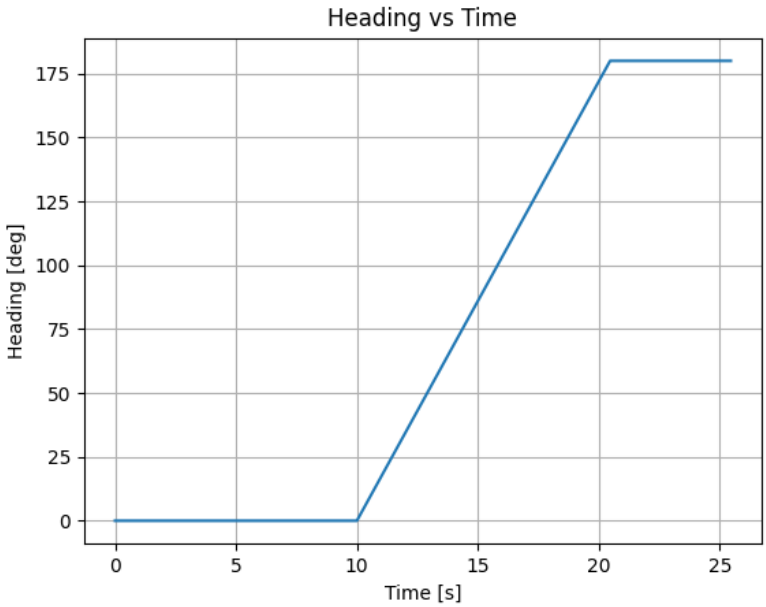


Figure 10: Plots of heading



Spiral test

Figure 11: Plots of Spiral

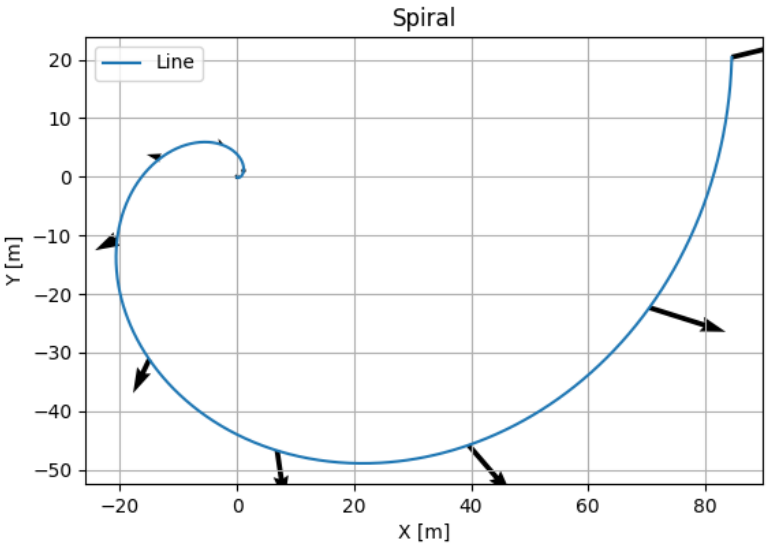


Figure 12: Plots of Spiral velocity